Sprint 5

Grupo 03
Alejandro De Haro, Juan Díaz & Daniel Escobosa

Tecnologías de Procesamiento Big Data
3º Grado en Ingeniería Matemática e Inteligencia Artificial

Índice

Tabla de contenido

Introducción *3*

Metodología *4*

Conclusión *8*

Introducción

Contexto

En los sprints anteriores del proyecto se diseñó e implementó una arquitectura Big Data orientada al almacenamiento, procesamiento y análisis de datos de criptomonedas utilizando servicios del ecosistema Amazon Web Services (AWS). En particular, se desarrolló un Data Lake basado en una arquitectura en tres capas (Bronce, Plata y Oro) que permite gestionar el ciclo completo del dato, desde su ingestión inicial hasta la generación de datasets analíticos enriquecidos con distintos indicadores técnicos.

Tal como se describió en los sprints previos, la arquitectura implementada se centraba principalmente en el procesamiento de datos históricos, permitiendo almacenar y transformar información procedente de fuentes externas para su posterior análisis. Este enfoque facilita la construcción de datasets estructurados y optimizados para el análisis financiero y la generación de conocimiento a partir de los datos.

No obstante, muchas aplicaciones relacionadas con el análisis de mercados financieros requieren también procesar información en tiempo real, ya que los precios de los activos y el volumen de transacciones cambian constantemente. Para abordar este tipo de escenarios resulta necesario incorporar tecnologías capaces de gestionar flujos de datos continuos, permitiendo capturar, distribuir y procesar eventos conforme se generan.

Descripción del problema y objetivos del sprint

El objetivo de este sprint es implementar y comprender un flujo básico de ingestión y consumo de datos en tiempo real utilizando Apache Kafka. Para ello, se trabaja con un broker Kafka remoto al que se accede mediante autenticación y con el que se interactúa tanto mediante herramientas de línea de comandos como mediante scripts desarrollados en Python.

En primer lugar, se utilizan programas de ejemplo para producir y consumir mensajes en un topic de Kafka, permitiendo verificar el funcionamiento del sistema de mensajería. Posteriormente, se integra una fuente de datos en tiempo real obtenida a través de la API de Binance, que proporciona información sobre el mercado de criptomonedas.

El objetivo final es construir un flujo en el que los datos obtenidos en tiempo real puedan ser publicados en un topic específico del broker Kafka (en nuestro caso, imat3a-AVAX) y posteriormente consumidos por diferentes procesos, permitiendo así su análisis o procesamiento posterior. De esta manera, la práctica permite comprender los conceptos fundamentales del streaming de datos y el papel que desempeña Kafka dentro de arquitecturas de procesamiento de eventos.

Metodología

En este sprint se ha desarrollado un flujo de ingestión y consumo de datos en tiempo real utilizando Apache Kafka y la API de Binance. La metodología seguida se ha estructurado en varias fases: configuración del entorno Kafka, validación del envío y recepción de mensajes, desarrollo de los scripts de integración con Binance y verificación del flujo de datos en tiempo real.

Configuración del entorno y acceso al broker Kafka

En primer lugar, se configuró el entorno de trabajo para poder interactuar con el broker Kafka remoto proporcionado para la práctica. Este broker requiere autenticación mediante el protocolo SASL_PLAINTEXT y el mecanismo PLAIN, utilizando las credenciales facilitadas.

Para ello se preparó un archivo de configuración client.properties, que permite a las herramientas de Kafka autenticarse correctamente contra el broker. Además, se configuraron las variables necesarias para definir el bootstrap server y poder ejecutar los comandos de administración y consumo de Kafka desde la terminal.

Una vez completada la configuración, se verificó la conexión con el broker listando los consumer groups existentes y comprobando la disponibilidad del sistema de mensajería.

```
[ec2-user@ip-172-31-20-180 ~]$ $KAFKA_HOME/bin/kafka-topics.sh   --bootstrap-server "$BOOTSTRAP"   --command-config client.properties   --list
Alvaro
Calvo
TOPIC_DE_JUAN_DIAZ_ARENAS_OFICIAL
__consumer_offsets
demo-topic
esther_lucia_maria
imat3a-DOGE
imat3a-DOT
imat3a-LINK
imat3a-TRX
imat3a_AAVE
imat3a_Avaxlos
imat3a_test
imat3a_xlm
imat3b-DOT
imat3b-XRP
jambo_calvo
prueba
prueba_big_mac
yetsky
```

Ilustración 1. Listado de consumer groups en Kafka

Creación y validación del flujo básico Producer–Consumer

Antes de integrar los datos de Binance, se implementó una prueba básica de comunicación entre un producer y un consumer de Kafka para validar el funcionamiento del sistema.

Para ello se utilizaron dos scripts en Python:

- kafka_simple_producer.py
- kafka_simple_consumer.py

El script producer se encarga de enviar un mensaje sencillo en formato JSON al topic imat3a-AVAX. Este mensaje contiene una clave (key1) y un valor estructurado con un campo de prueba. El script consumer se conecta al mismo topic y lee los mensajes disponibles, mostrando información relevante como la clave del mensaje, el valor, el offset y el timestamp generado por Kafka.

La ejecución del productor permitió enviar correctamente el mensaje al broker Kafka.

```
[ec2-user@ip-172-31-20-180 ~]$ python3 kafka_simple_producer.py
Mensaje a enviar:  key1 {'field1': 'value1'}
[ec2-user@ip-172-31-20-180 ~]$
```

Ilustración 2. Ejecución de kafka_simple_producer.py

Posteriormente, al ejecutar el consumidor se pudo comprobar que el mensaje era recuperado correctamente desde el topic, validando así el flujo de comunicación entre productor y consumidor.

```
[ec2-user@ip-172-31-20-180 ~]$ python3 kafka_simple_consumer.py
TopicPartition(topic='imat3a-AVAX', partition=0)
key:       key1
value:     {'field1': 'value1'}
offset:    1
timestamp: 1773255175697
[ec2-user@ip-172-31-20-180 ~]$
```

Ilustración 3. Ejecución de kafka_simple_consumer.py

Esta primera fase permitió confirmar que la configuración de Kafka y la comunicación con el broker funcionaban correctamente antes de integrar la fuente de datos en tiempo real.

Monitorización de topics y consumer groups

Una vez verificado el flujo básico de mensajes, se utilizaron herramientas de línea de comandos de Kafka para monitorizar el estado de los consumer groups y los offsets de los topics.

Mediante el comando kafka-consumer-groups.sh --describe se pudo observar el estado del grupo de consumidores utilizado en la práctica (imat3a_group03), así como la posición de lectura de cada partición y el lag existente entre los mensajes producidos y consumidos.

Esta información resulta fundamental para entender el funcionamiento interno de Kafka y comprobar si los consumidores están procesando correctamente los mensajes generados por los productores.

```
[ec2-user@ip-172-31-20-180 ~]$ $KAFKA_HOME/bin/kafka-consumer-groups.sh \
  --bootstrap-server "$BOOTSTRAP" \
  --command-config client.properties \
  --describe \
  --group imat3a_groupAvax

Consumer group 'imat3a_groupAvax' has no active members.

GROUP            TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID     HOST            CLIENT-ID
imat3a_groupAvax imat3a_Avaxlos  0          4               4               0               -               -               -[ec2-user@ip-172-31-20-180 ~]$
[ec2-user@ip-172-31-20-180 ~]$ $KAFKA_HOME/bin/kafka-consumer-groups.sh   --bootstrap-server "$BOOTSTRAP"   --command-config client.properties   --describe   --group
imat3a_group1

GROUP           TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID                                          HOST            CLIEN
T-ID
imat3a_group1   imat3a-DOT      0          2               2               0               console-consumer-02394340-1015-4156-bbaa-fb3f2fc9b247 /51.94.215.12   conso
le-consumer
imat3a_group1   imat3a_test     0          15              17              2               -                                                    -               -
imat3a_group1   imat3a-DOGE     0          3               3               0               -                                                    -               -
imat3a_group1   imat3a_BTC      0          2               2               0               -                                                    -               -
imat3a_group1   imat3a_Avaxlos  0          3               4               1               -                                                    -               -[ec2
-user@ip-172-31-20-180 ~]$
```

Ilustración 4. Descripción del consumer group y offsets

Captura de datos en tiempo real desde Binance

Una vez validado el funcionamiento del sistema de mensajería, se implementó la captura de datos de mercado en tiempo real utilizando la API de Binance.

Para ello se utilizó la librería oficial de Binance para Python, que permite establecer conexiones WebSocket mediante la clase ThreadedWebsocketManager. A través de esta conexión se reciben actualizaciones continuas de las velas de mercado (candlesticks) para la criptomoneda seleccionada.

Inicialmente se utilizó el script binance_real_time.py para comprobar que la conexión con la API funcionaba correctamente y que se recibían datos de mercado en tiempo real.

Este script muestra por pantalla información básica de cada vela, incluyendo el intervalo temporal, el precio de cierre y el volumen de negociación.

Con este script mandamos la información cada vez que se cierra una vela, es clave mencionar el cambio del formato temporal, este se devuelve tanto en ms como en utc.

Integración de Binance con Kafka

Una vez validada la recepción de datos desde Binance, se desarrolló el script binance_to_kafka.py, cuyo objetivo es integrar la fuente de datos en tiempo real con el sistema de mensajería Kafka.

Este script realiza las siguientes tareas:

1. Establece una conexión WebSocket con Binance para recibir velas de mercado de la criptomoneda AVAXUSDT con un intervalo de 1 minuto.
2. Procesa cada mensaje recibido y filtra únicamente las velas que han finalizado (x = True).
3. Extrae los datos relevantes del mensaje:
    a. símbolo de la criptomoneda
    b. precio de cierre
    c. volumen
    d. timestamp de cierre
4. Convierte el timestamp proporcionado por Binance al formato estándar UTC ISO8601.
5. Construye un mensaje JSON estructurado.
6. Envía el mensaje al topic imat3a-AVAX de Kafka utilizando un KafkaProducer.

Durante la ejecución del script se muestra por consola el contenido de cada mensaje enviado al broker Kafka.

```
[ec2-user@ip-172-31-20-180 ~]$ python3 binance_to_kafka.py
Escuchando Binance para AVAXUSDT y publicando en imat3a-AVAX
Pulsa ENTER para salir...
Enviando a Kafka...
key=AVAXUSDT value={'symbol': 'AVAXUSDT', '@timestamp': '2026-03-12T17:36:59Z', 'close': 9.57, 'volume': 510.51, 'timestamp_ms': 1773337019999}
```

Ilustración 5. Ejecución de binance_to_kafka.py enviando datos a Kafka

Consumo de datos en tiempo real desde Kafka

Finalmente, para comprobar que los datos generados por Binance estaban siendo correctamente enviados al broker Kafka, se desarrolló el script kafka_real_time_consumer.py.

Este consumidor se conecta al topic imat3a-AVAX y permanece escuchando continuamente nuevos mensajes publicados. Cada vez que se recibe un mensaje, el script muestra información detallada sobre el mismo, incluyendo:

- Topic
- Partición
- Offset
- Clave del mensaje
- Contenido del mensaje
- Timestamp

De esta forma es posible observar en tiempo real cómo los datos procedentes de Binance son capturados por el productor y posteriormente consumidos desde Kafka.

```
--------------------------------------------------
Nuevo mensaje recibido
topic:      imat3a-AVAX
partition:  0
offset:     21
key:        AVAXUSDT
value:      {'symbol': 'AVAXUSDT', '@timestamp': '2026-03-12T17:55:59Z', 'close': 9.54, 'volume': 525.75, 'timestamp_ms': 1773338159999}
timestamp:  1773338160196
--------------------------------------------------
Nuevo mensaje recibido
topic:      imat3a-AVAX
partition:  0
offset:     22
key:        AVAXUSDT
value:      {'symbol': 'AVAXUSDT', '@timestamp': '2026-03-12T17:56:59Z', 'close': 9.54, 'volume': 140.05, 'timestamp_ms': 1773338219999}
timestamp:  1773338220207
--------------------------------------------------
Nuevo mensaje recibido
topic:      imat3a-AVAX
partition:  0
offset:     23
key:        AVAXUSDT
value:      {'symbol': 'AVAXUSDT', '@timestamp': '2026-03-12T17:57:59Z', 'close': 9.55, 'volume': 456.08, 'timestamp_ms': 1773338279999}
timestamp:  1773338280232
--------------------------------------------------
Nuevo mensaje recibido
topic:      imat3a-AVAX
partition:  0
offset:     24
key:        AVAXUSDT
value:      {'symbol': 'AVAXUSDT', '@timestamp': '2026-03-12T17:58:59Z', 'close': 9.55, 'volume': 1325.45, 'timestamp_ms': 1773338339999}
timestamp:  1773338340251
--------------------------------------------------
```

Ilustración 6. Ejecución de kafka_real_time_consumer.py recibiendo datos en tiempo real

Conclusión

En este sprint se ha abordado la incorporación de procesamiento de datos en tiempo real dentro de la arquitectura Big Data desarrollada en los sprints anteriores. Mientras que las fases previas del proyecto se centraban en el almacenamiento, transformación y análisis de datos históricos mediante un Data Lake en AWS, en este sprint se ha introducido una arquitectura basada en streaming de eventos utilizando Apache Kafka.

Para ello, se ha implementado un flujo de datos que permite capturar información de mercado en tiempo real mediante la API de Binance, concretamente a través de conexiones WebSocket que proporcionan actualizaciones continuas sobre el precio y volumen de criptomonedas. Estos datos son posteriormente enviados a un broker Kafka, donde se publican en el topic imat3a-AVAX, permitiendo su distribución a distintos consumidores.

Asimismo, se han utilizado productores y consumidores desarrollados en Python para enviar y leer mensajes desde Kafka, lo que ha permitido comprender el funcionamiento de los componentes fundamentales de esta tecnología, como los topics, los consumer groups y la gestión de mensajes en un sistema de mensajería distribuido.

En conjunto, este sprint demuestra cómo una arquitectura de datos puede ampliarse para integrar fuentes de información en tiempo real, complementando el procesamiento batch desarrollado anteriormente. La utilización de Kafka permite desacoplar la generación y el consumo de datos, facilitando la construcción de sistemas escalables capaces de gestionar flujos continuos de información procedentes de mercados financieros u otras fuentes dinámicas.